

GUIÓN OFICIAL DE DEFENSA EN VIDEO (EQUIPO DE 4 INTEGRANTES)

Universidad Nacional Experimental de Guayana (UNEG)

Vicerrectorado Académico — Coordinación General de Pregrado

Lenguajes y Compiladores - Sección 01 (Prof. Msc. Félix Márquez)

Tema 4: Análisis Léxico — Autómatas Finitos/Pila y Metacompiladores (**Flex**)

Duración Total Estimada: ~10 Minutos (2 min 30 s por cada integrante)

Modalidad: Exposición grabada (Defensa 10%)

Consejos Generales para la Grabación del Video

1. **Calidad de Sonido y Nitidez:** Utilicen auriculares o micrófono externo en un lugar silencioso. Verifiquen que la presentación en HTML con estilo *Claymorphism* se proyecte en pantalla completa (**F11**) a resolución 1080p y con texto perfectamente legible.
 2. **Postura y Cámara:** Mantengan la cámara encendida en una ventana (o esquina superior/inferior) mientras comparten pantalla. Hablen con naturalidad, fluidez y seguridad profesional.
 3. **Manejo del Tiempo:** Practiquen con cronómetro para que cada intervención dure exactamente entre **2 minutos 15 segundos y 2 minutos 30 segundos**, sumando el total ideal de 10 minutos exigido por la evaluación.
-

INTEGRANTE 1: FERNANDO CENTENO — INTRODUCCIÓN Y AUTÓMATAS DE PILA (PDA)

- **Duración:** 0:00 - 2:30 (~2 min 30 s)
- **Láminas Asignadas en la Presentación HTML:** Diapositiva 1 (Portada), Diapositiva 2 (Introducción al Análisis Léxico) y Diapositiva 3 (Autómatas de Pila vs. AFD).

[0:00 - 0:35] Saludo e Introducción General

Fernando Centeno:

"¡Saludos, estimado profesor Msc. Félix Márquez, y compañeros! Nosotros somos el Equipo de Trabajo del Tema 4 de Lenguajes y Compiladores, integrado por mis compañeros Juan Longart, Adrian Reina, Rafael Rodriguez y mi persona, Fernando Centeno. Hoy les presentaremos el desarrollo teórico y práctico de nuestro proyecto sobre *Análisis Léxico, Autómatas de Pila y Metacompiladores Flex*."

"Como sabemos, el Analizador Léxico (o *Lexer*) constituye la puerta de entrada de todo compilador. Su trabajo fundamental es la *Tokenización*: leer el programa fuente carácter por carácter, verificar la pertenencia al alfabeto del lenguaje y agrupar los caracteres en unidades lógicas llamadas *Tokens*, eliminando elementos no sintácticos como comentarios o espacios en blanco."

[0:35 - 1:40] Teoría del Autómata de Pila y Definición Formal

(Pasar a Diapositiva 3 - Autómatas de Pila)

Fernando Centeno:

"Para comprender el alcance del análisis léxico, debemos estudiar la jerarquía de las máquinas teóricas. En nuestra Actividad 1 investigamos el *Autómata de Pila (PDA)*. Formalmente, un autómata de pila se define como una tupla de 7 elementos: ' $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ '. "

"A diferencia de un Autómata Finito Determinístico (AFD) —que posee una memoria estrictamente fija limitada por su número de estados y por ende solo reconoce *Lenguajes Regulares*—, el Autómata de Pila incorpora una *memoria infinita con estructura LIFO* (Último en entrar, primero en salir). Esta pila le otorga la capacidad computacional necesaria para reconocer *Lenguajes Libres de Contexto (GLC)* según la Jerarquía de Chomsky."

[1:40 - 2:30] Ejemplos Clave y Conclusión del Poder Computacional

Fernando Centeno:

"Un ejemplo clásico que demuestra la superioridad de la pila es el lenguaje de cadenas balanceadas ' $L = \{a^n b^n \mid n \geq 1\}$ '. Un autómata finito es incapaz de recordar cuántas ' a ' leyó si ' n ' es arbitrariamente grande; el autómata de pila, en cambio, apila un símbolo por cada ' a ' y desapila por cada ' b ', aceptando por estado final o pila vacía únicamente si hay un emparejamiento perfecto."

"En conclusión, el AFD es el motor ultrarrápido y perfecto para el *Análisis Léxico* (tokenizar palabras e identificadores), mientras que el Autómata de Pila es el corazón indispensable del *Análisis Sintáctico (Parser)*, ya que permite validar estructuras anidadas como paréntesis, bloques de llaves '{ }' y sentencias recursivas. A continuación, dejo a mi compañero Juan Longart, quien presentará nuestro Lexer desarrollado desde cero para archivos Docker."



INTEGRANTE 2: JUAN LONGART — ANALIZADOR LÉXICO PARA DOCKERFILE EN PYTHON (DESDE CERO)

- **Duración:** 2:30 - 5:00 (~2 min 30 s)
- **Láminas Asignadas en la Presentación HTML:** Diapositiva 4 (Arquitectura del Lexer en Python) y Diapositiva 5 (Demostración de Ejecución y Manejo de Errores).

[2:30 - 3:15] Introducción al Reto de Ingeniería y Diseño de la Gramática

(Pasar a Diapositiva 4 - Lexer Dockerfile en Python)

Juan Longart:

"¡Muchas gracias, Fernando! Para la Actividad 2, asumimos el reto de construir un analizador léxico *completamente desde cero* utilizando Python para verificar la sintaxis de archivos *Dockerfile*. Es muy importante aclarar que *no se requiere tener instalado Docker* para ejecutar este programa, ya que nuestro analizador evalúa directamente la estructura gramatical del texto fuente."

"Diseñamos un alfabeto léxico organizado por precedencia exacta en una lista de tuplas con expresiones regulares. Clasificamos las directivas clave como **FROM**, **RUN**, **CMD**, **COPY**, **ENV** y **EXPOSE** bajo el token **DIRECTIVE**; también capturamos flags con doble guion (**--platform**, **--from**) bajo **FLAG**, variables de entorno (**\$PORT** o **\${VAR}**) como **VARIABLE**, números, puertos (**8080/tcp**) y cadenas de texto."

[3:15 - 4:05] Explicación del Motor con Generadores (**re.finditer**)

Juan Longart:

"Para la arquitectura interna, evitamos bucles lentos de división de cadenas y construimos un motor basado en la función **finditer** del módulo **re** de Python con grupos nombrados (**(?P<name>pattern)**). Nuestro generador (**dockerfile_lexer**) recorre el archivo en una sola pasada de complejidad lineal **\$O(n)\$**."

"A medida que lee, mantiene un seguimiento geométrico riguroso de la posición: cada vez que reconoce un salto de línea (**NEWLINE**), incrementa el contador de líneas y reinicia el desplazamiento de columnas, permitiendo rastrear exactamente dónde se encuentra cada palabra."

[4:05 - 5:00] Pruebas, Salida en Tabla y Detección de Errores (**MISMATCH**)

(Pasar a Diapositiva 5 - Demostración de los 3 Ejemplos)

Juan Longart:

"Para validar nuestro sistema, creamos 3 ejemplos de prueba y un script automatizado **run_dockerfile_lexer.py**:

1. **Ejemplo 1:** Un **Dockerfile** estándar de Node.js, donde el lexer procesa las 36 directivas exitosamente declarando el estado como **ACEPTADO**.
2. **Ejemplo 2:** Un build multi-etapa (*multi-stage*) complejo en Go, reconociendo 65 tokens incluyendo directivas avanzadas como **HEALTHCHECK** y **ARG**.
3. **Ejemplo 3 (Prueba de error léxico):** Introdujimos deliberadamente un símbolo ilegal —el signo de interrogación invertido '¿'— en la directiva **RUN**. El autómata detuvo inmediatamente la lectura al caer en nuestra regla de excepción (**MISMATCH**) e imprimió un reporte pedagógico en pantalla:

```
[!] ERROR LÉXICO en línea 7, columna 28: Carácter o secuencia no reconocida '¿', protegiendo al compilador de procesar código malformado. A continuación, mi compañero Adrian Reina explicará la construcción mediante metacompiladores."
```



INTEGRANTE 3: ADRIAN REINA — SUBCONJUNTO DE RUST (**MINIRUST**) CON METACOMPILADOR **FLEX**

- **Duración:** 5:00 - 7:30 (~2 min 30 s)
- **Láminas Asignadas en la Presentación HTML:** Diapositiva 6 (Manual de Usuario del Metacompilador Flex) y Diapositiva 7 (Lenguaje MiniRust y Código **.1**).

[5:00 - 5:45] Manual de Usuario de Flex y Estructura en 3 Secciones

(Pasar a Diapositiva 6 - Metacompilador Flex)

Adrian Reina:

"¡Saludos a todos! Para la Actividad 3, pasamos de la programación manual a la automatización industrial utilizando Flex (Fast Lexical Analyzer Generator), el metacompilador más difundido del mundo C/UNIX."

"Flex recibe un archivo de especificación declarativa con extensión `.l` y genera automáticamente un archivo de código C llamado `lex.yy.c`, el cual contiene un Autómata Finito Determinístico altamente optimizado en memoria. Todo archivo `.l` se divide estrictamente en tres secciones separadas por el símbolo `%%`:

1. **Sección de Definiciones** (`{ ... %}`): Aquí incluimos librerías C, contadores globales de línea/columna y macros de expresiones regulares.
2. **Sección de Reglas**: Donde emparejamos cada patrón o regex con un bloque de código C que se ejecutará al coincidir.
3. **Sección de Código de Usuario**: Donde implementamos la función `main()` que llama a la función principal autogenerada `yyllex()`.`**`

[5:45 - 6:40] Diseño del Lenguaje `MiniRust`

(Pasar a Diapositiva 7 - Especificación de MiniRust)

Adrian Reina:

"Para probar Flex, diseñamos `MiniRust`, un subconjunto estructural del moderno lenguaje Rust. Definimos soporte para los tipos de datos primitivos `i32`, `f64`, `bool`, `char` y `String`; palabras reservadas como `fn`, `let`, `mut`, `if`, `else`, `while`, `for`, `return` y la macro `println!`; junto con operadores multicarácter relacionales (`==`, `!=`, `<=`, `>=`), lógicos (`&&`, `||`) y flechas de retorno (`->`)."

[6:40 - 7:30] Compilación, Precedencia y Emulador en Python

Adrian Reina:

"En nuestro archivo `rust_lexer.l`, aplicamos la regla fundamental de Flex: la coincidencia más larga (Longest Match) y el orden de declaración. Colocamos las palabras reservadas antes que la regla general de identificadores (`[a-zA-Z][a-zA-Z0-9_]*`) para que la variable `let` sea reconocida exactamente como `TK_KEYWORD_LET` y no como un nombre cualquiera."

"Para facilitar la revisión en cualquier sistema operativo, no solo entregamos el código C autogenerado, sino también un módulo emulador `rust_lexer_py.py` que tokeniza el código `MiniRust` con el mismo estándar, probándolo exitosamente con programas de cálculo factorial y validación de errores. Ahora, mi compañero Rafael Rodriguez cerrará con una indagación crucial sobre Flex y la Seguridad Informática."



INTEGRANTE 4: RAFAEL RODRIGUEZ — FLEX EN CIBERSEGURIDAD Y CONCLUSIONES GENERALES

- **Duración:** 7:30 - 10:00 (~2 min 30 s)
- **Láminas Asignadas en la Presentación HTML:** Diapositiva 8 (Flex en Ciberseguridad e IDS/WAF) y Diapositiva 9 (Conclusiones Finales del Equipo).

[7:30 - 8:20] Indagación: ¿Por qué Flex es el motor de la Ciberseguridad?

(Pasar a Diapositiva 8 - Flex en Seguridad Informática)

Rafael Rodriguez:

"¡Excelente explicación, Adrian! Para finalizar nuestro proyecto en la Actividad 4, indagamos y reflexionamos sobre una de las aplicaciones industriales más fascinantes y menos conocidas de los autómatas y de **Flex: la Seguridad Informática (Ciberseguridad)**."

"En un centro de operaciones de seguridad (SOC) o en un cortafuegos perimetral, los sistemas deben inspeccionar flujos de tráfico de red masivos a velocidades de 10 o 100 Gigabits por segundo. Si un Sistema de Detección de Intrusos (IDS) utilizara bucles interpretados lentos o verificaciones ingenuas de cadenas para revisar miles de firmas de ataques, colapsaría la red. Gracias a que Flex genera un autómata determinístico puro de complejidad lineal $O(n)$, permite analizar cada byte del tráfico en tiempo real con latencia prácticamente nula."

[8:20 - 9:15] Casos de Uso Reales: Snort, ModSecurity y YARA

Rafael Rodriguez:

"Encontramos la aplicación directa de Flex en tres grandes estándares mundiales de seguridad:

1. **Snort y Suricata (IDS/IPS):** Utilizan analizadores léxicos para tokenizar a máxima velocidad las cabeceras de paquetes IP/TCP/UDP y emparejar opciones de reglas como `sid:100001` o firmas binarias en el payload.
2. **ModSecurity (WAF - Web Application Firewall):** Tokeniza peticiones HTTP para detectar sintaxis maliciosa de inyección SQL (`SQLi`) o *Cross-Site Scripting* (`XSS`) antes de que toquen el servidor.
3. **Reglas YARA (Caza de Malware):** YARA es la herramienta estándar forense para identificar troyanos y virus en memoria o disco. Su código fuente oficial (`libyara`) utiliza **Flex y Bison** para compilar las reglas de los analistas (`rule`, `meta`, `strings`, `condition`) y sus patrones hexadecimales (`{ E8 ?? ?? 8B 45 }`). Como aporte extra, desarrollamos en nuestro repositorio el archivo `yara_lexer_demo.1`, demostrando un autómata funcional capaz de tokenizar reglas de malware en C."**

[9:15 - 10:00] Conclusiones Finales y Despedida

(Pasar a Diapositiva 9 - Conclusiones)

Rafael Rodriguez:

"En conclusión general:

1. El análisis léxico es la base insustituible para purificar y validar el código fuente antes del análisis sintáctico.
2. Los Autómatas Finitos dominan la tokenización por su velocidad y bajo costo de memoria, mientras que los Autómatas de Pila reinan en la sintaxis para balancear bloques anidados.
3. La programación manual desde cero otorga personalización extrema en el reporte de errores, mientras que los metacompiladores como Flex garantizan robustez y rendimiento de grado industrial en lenguajes complejos y motores de defensa cibernética."**

"Todo el código fuente en Python, Flex (`.1`), C y MiniRust (`.rs`), junto con el informe técnico completo, están disponibles en nuestro repositorio GitHub para su revisión. ¡Muchísimas gracias

por su atención, profesor Msc. Félix Márquez, quedamos atentos a cualquier pregunta!"